

Problem Set 2

Peter Shi, Wenhui Xu

June 21, 2026

Question 1

Consider the neoclassical growth model you solved in Problem 3 of HW1 with all the same details, except that we will now focus on the following specific assumptions: Set $\beta = 0.96$, (and keep $\alpha = 0.4$, $\delta = 0.1$) and use a tolerance value of 106 based on the decision rule as described in the last homework. When constructing the capital grid, take the maximum capital value k_N to be equal to 10 times the steady-state capital level (which you need to compute). And you should figure out what value makes sense for the lowest capital level k_0 of your grid. You will solve the problem for two different capital grids, one with 101 points and another with 1001 points, and will compare. All other details are the same.

Preliminaries:

We define, **production function**:

$$f(k) = Ak^\alpha + (1 - \delta)k, A > 0, 0 < \alpha < 1, 1 \geq \delta \geq 0$$

whatever is not consumed are saved into next period, as capital, thus, we define the **policy function**:

$$g(k) = k' = f(k) - c$$

log utility derived from consumption: $U(c) = \log(c)$, can also be described as a path of k_i

$$U(c) = \log(Ak^\alpha + (1 - \delta)k - k')$$

The **objective** is

$$V(k) = \max_{k'} \{ \log(Ak^\alpha + (1 - \delta)k - k') + \beta V(k') \}$$

```
u(c) = (c > 0) ? log(c) : -Inf #define log utility  
  
f(k; A=1.0, alpha=0.4, delta=0.1) = A * k^alpha + (1.0 - delta) * k #define  
production function
```

We start from the deterministic capital accumulation problem with log utility. Define the resource function

$$f(k) = Ak^\alpha + (1 - \delta)k.$$

Given current capital k and a choice of next-period capital k' , consumption is determined by

$$c = f(k) - k'.$$

The Bellman equation is

$$V(k) = \max_{k' \in K} \{ \log(f(k) - k') + \beta V(k') \}.$$

The optimal policy (decision rule) is the maximizer

$$g(k) = \arg \max_{k' \in K} \{ \log(f(k) - k') + \beta V(k') \},$$

so that under the optimal policy $k' = g(k)$.

To construct a grid for k , we use the steady state k^* as an anchor. In a steady state, the optimal policy keeps capital constant:

$$k' = k = k^*.$$

The Euler equation for this problem is

$$u'(c_t) = \beta u'(c_{t+1}) (\alpha A k_{t+1}^{\alpha-1} + 1 - \delta).$$

With log utility $u(c) = \log c$, we have $u'(c) = 1/c$. In steady state, $c_t = c_{t+1} = c^*$ and $k_{t+1} = k^*$, so the Euler equation becomes

$$\frac{1}{c^*} = \beta \frac{1}{c^*} (\alpha A (k^*)^{\alpha-1} + 1 - \delta).$$

Cancelling $1/c^*$ yields

$$1 = \beta (\alpha A (k^*)^{\alpha-1} + 1 - \delta).$$

Rearranging,

$$\frac{1}{\beta} - (1 - \delta) = \alpha A (k^*)^{\alpha-1},$$

and therefore

$$(k^*)^{\alpha-1} = \frac{\frac{1}{\beta} - (1 - \delta)}{\alpha A}.$$

Finally,

$$k^* = \left(\frac{\frac{1}{\beta} - (1 - \delta)}{\alpha A} \right)^{\frac{1}{\alpha-1}}.$$

We then choose grid bounds proportional to k^* , for example

$$k_{\min} = \underline{\kappa} k^*, \quad k_{\max} = \bar{\kappa} k^*,$$

with constants such as $\underline{\kappa} = 0.25$ and $\bar{\kappa} = 1.75$, and create an evenly spaced grid on $[k_{\min}, k_{\max}]$. Now, define a function to generate grid space parameters. **Set the maximum capital value equal to 10 times the steady-state capital level.**

```
function kstar_and_grid(; beta=0.96, A=1.0, alpha=0.4, delta=0.1, k_low=0.25
    , k_high=10, N=11)

# steady-state exists if (1/β - (1-δ)) > 0
rhs = (1/beta - (1 - delta)) / (alpha * A)
@assert rhs > 0 "Steady-state formula requires (1/β - (1-δ)) / (αA) > 0."

kstar = rhs^(1/(alpha - 1))

kmin = k_low * kstar
kmax = k_high * kstar
@assert kmin > 0 && kmax > kmin

kgrid = range(kmin, kmax; length=N)

return kstar, kmin, kmax, kgrid
end
```

Question a

Solve the model using Howard's Modified Policy Iteration (MPI) algorithm, for $m = 5, 25$, and 500 Howard steps. Plot and compare both the solution (value function and decision rules) and the time it takes to obtain a solution to the basic VFI. ***(For MA and PhDs:)** Solve also for $m = 10$ and 100.

```
function standard_vfi(;beta=0.96, A=1.0, alpha=0.4, delta=0.1, N=11, k_low=0.25, k_high=10, tol=1e-6, maxit=500000, verbose = true)

# Step 0: grid (make sure it uses the same parameters)
kstar, kmin, kmax, kgrid = kstar_and_grid(;beta=beta, A=A, alpha=alpha,delta=delta,k_low=k_low, k_high=k_high, N=N)
@assert length(kgrid) == N
@show first(kgrid) last(kgrid) step(kgrid)
# Step 1: initial guess V0
V = zeros{Float64, N} #vector length of 11, the up-to-date value functions
policy_idx = fill(1, N) #the up-to-date policy functions
j0 = argmin(abs.(kgrid .- kstar)) # index of grid point closest to kstar
policy_idx .= j0
policy_idx_new = similar(policy_idx) # the current policy functions
V_new = similar(V) #vector length of 11 the current value functions
```

```

it = 0
converged = false

while it < maxit

    it += 1

    # Step 2(i): maximization step
    dist = 0.0
    for i in 1:N
        k = kgrid[i]

        best_val = -Inf # Initialized to -Inf to correctly capture negative
                        # utility values
        best_j = 1

        for j in 1:N
            kp = kgrid[j]
            c = f(k, A=A, alpha=alpha, delta=delta) - kp
            val = u(c) + beta * V[j] # u(c) should be -Inf if c<=0

            if val > best_val
                best_val = val
                best_j = j
                #println(val)
                #println(best_val)
            end
        end

        policy_idx_new[i] = best_j

        # Step 2(ii): evaluation step, for each i, update once
        V_new[i] = best_val

        # Step 3: stopping rule check, for each i, update once
        dist = max(dist, abs(kgrid[policy_idx_new[i]] - kgrid[policy_idx[i]]
        ))
        #println("dist is ", dist)
    end

    # update for next iteration
    V.= V_new
    policy_idx .= policy_idx_new

    if dist <= tol
        converged = true
        if verbose
            println("STOP at it=$it with dist=$dist")
        end
        break
    end

end

if it == maxit
    converged = false
end

if it % 50 == 0 && verbose
    println("it=$it, dist=$dist, policy_idx[1:5]=", policy_idx[1:5])
end

```

```

end

g = kgrid[policy_idx]
return (; V, g, kgrid, policy_idx, it, converged, kstar, kmin, kmax)
end

```

```

function howard_vfi(;beta=0.96, A=1.0, alpha=0.4, delta=0.1, N=11, k_low=0.2
    5, k_high=10, tol=1e-6, maxit=500000, howard_it = 20, verbose = true)

# Step 0: grid (make sure it uses the same parameters)
kstar, kmin, kmax, kgrid = kstar_and_grid(;beta=beta, A=A, alpha=alpha,delta
    =delta,k_low=k_low, k_high=k_high, N=N)
@assert length(kgrid) == N
@show first(kgrid) last(kgrid) step(kgrid)
# Step 1: initial guess V0
V = zeros(Float64, N) #vector length of 11, the up-to-date value functions
policy_idx = fill(1, N) #the up-to-date policy functions
j0 = argmin(abs.(kgrid .- kstar)) # index of grid point closest to kstar
policy_idx .= j0
policy_idx_new = similar(policy_idx) # the current policy functions
V_new = similar(V) #vector length of 11 the current value functions

it = 0
converged = false

while it < maxit

    it += 1

    # Step 2(i): maximization step
    dist = 0.0
    for i in 1:N
        k = kgrid[i]

        best_val = -Inf # Initialized to -Inf to correctly capture negative
            utility values
        best_j = 1

        for j in 1:N
            kp = kgrid[j]
            c = f(k, A=A, alpha=alpha, delta=delta) - kp
            val = u(c) + beta * V[j] # u(c) should be -Inf if c<=0

            if val > best_val
                best_val = val
                best_j = j
                #println(val)
                #println(best_val)
            end
        end
        policy_idx_new[i] = best_j
    end
end

```

```

        # Step 2(ii): evaluation step, for each i, update once
        V_new[i] = best_val

        # Step 3: stopping rule check, for each i, update once
        dist = max(dist, abs(kgrid[policy_idx_new[i]] - kgrid[policy_idx[i]]
        ))
        #println("dist is ", dist)
    end

    # update for next iteration
    V.= V_new
    policy_idx .= policy_idx_new

    # HPI
    for h in 1:howard_it
        for i in 1:N
            j = policy_idx[i]
            k = kgrid[i]
            kp = kgrid[j]

            c = f(k) - kp
            V_new[i] = u(c) + beta * V[j]
        end
        V .= V_new
    end

    if dist <= tol
        converged = true
        if verbose
            println("STOP at it=$it with dist=$dist")
        end
        break
    end
end

if it == maxit
    converged = false
end

if it % 50 == 0 && verbose
    println("it=$it, dist=$dist, policy_idx[1:5]=", policy_idx[1:5])
end

g = kgrid[policy_idx]
return (; V, g, kgrid, policy_idx, it, converged, kstar, kmin, kmax)
end

```

Runtime and convergence comparison:

The runtime comparison results show that basic value function iteration (VFI) becomes much slower when the capital grid is finer. With $N = 101$, VFI converges in 31 iterations and takes about 0.017 seconds. When the grid increases to $N = 1001$, VFI needs 64 iterations and the runtime rises sharply to about 3.55 seconds. This large increase is expected because VFI performs a full grid search over all candidate k' values at every state point, so the computational cost grows quickly as N increases.

Howard’s Modified Policy Iteration (MPI) provides clear speed improvements compared to basic VFI, especially for the large grid. For $N = 101$, MPI reduces the number of outer iterations from 31 to 17 ($m = 5$), 15 ($m = 10$), and 14 ($m = 25/100/500$), and the runtime drops to around 0.009–0.010 seconds for $m = 5$ to $m = 25$. The best timing in this case is $m = 10$ (0.00936 seconds), which corresponds to about a $1.83\times$ speedup. When m becomes large, MPI becomes slower: $m = 100$ is roughly the same speed as VFI, and $m = 500$ is much slower (only $0.31\times$ speedup). This happens because very large m adds too much policy evaluation work inside each iteration.

For $N = 1001$, MPI improves performance even more. MPI reduces outer iterations from 64 to 23 ($m = 5$) and to 20 ($m = 10/25/100/500$). Runtime falls from 3.55 seconds (VFI) to about 1.12–1.14 seconds for $m = 10$ and $m = 25$, giving speedups around $3.16\times$ and $3.12\times$. Again, very large m introduces extra overhead: $m = 500$ is slower than $m = 10$ or $m = 25$ (1.82 seconds, about a $1.95\times$ speedup). Overall, these results suggest there is a trade-off in choosing m . A moderate number of Howard steps (here $m = 10$ or $m = 25$) gives the best balance between fewer outer iterations and not spending too much time on extra evaluation.

Table 1: Performance Comparison of VFI and MPI

Method	Setting	Iterations	Time (seconds)	Speedup compared to baseline VFI
N = 101				
VFI	baseline	31	0.01712	1.000
MPI	m=5	17	0.01002	1.709
MPI	m=10	15	0.00936	1.829
MPI	m=25	14	0.01020	1.678
MPI	m=100	14	0.01740	0.984
MPI	m=500	14	0.05589	0.306
N = 1001				
VFI	baseline	64	3.54695	1.000
MPI	m=5	23	1.28489	2.760
MPI	m=10	20	1.12224	3.161
MPI	m=25	20	1.13784	3.117
MPI	m=100	20	1.24469	2.850
MPI	m=500	20	1.81744	1.952

Value and Policy Function Comparison:

Figures 1 and 2 compare the numerical solutions from basic VFI and Howard's MPI for two grid sizes ($N = 101$ and $N = 1001$). Overall, the figures suggest that all methods converge to essentially the same economic solution, and the main difference across methods is speed rather than the final policy.

First, the value function $V(k)$ is increasing and concave in k . This matches the economic intuition of the neoclassical growth model with log utility: having more capital allows higher consumption today and in the future, so lifetime utility increases, but with diminishing marginal returns, the curve becomes flatter as k grows. The value functions produced by MPI (for different Howard steps m) almost overlap with each other, and they are also very close to the basic VFI result. This indicates that changing m mainly affects convergence speed and does not materially change the final solution.

Second, the policy function $g(k)$ is increasing in k , which means higher current capital leads to higher optimal next-period capital. In both $N = 101$ and $N = 1001$, the policy curves lie below the 45-degree line ($k' = k$) over most of the range. This implies that the optimal choice typically sets $k' < k$ on these grid values, i.e., the agent consumes part of current resources rather than keeping capital constant. The step-like appearance is expected because the choice set for k' is restricted to discrete grid points, so the optimal policy moves in small jumps instead of being perfectly smooth.

Finally, comparing $N = 101$ and $N = 1001$, the solutions look very similar, but the $N = 1001$ plots are smoother (especially for the policy function). This is because a finer grid provides a more detailed approximation to the continuous problem. The fact that the solutions are stable across grid sizes suggests the numerical results are not driven by a particular grid choice.

In summary, the plots show that MPI with different m values and basic VFI deliver essentially the same value and policy functions. The key benefit of MPI is faster convergence (as shown in the runtime table), while the final computed decision rule is very similar across methods.

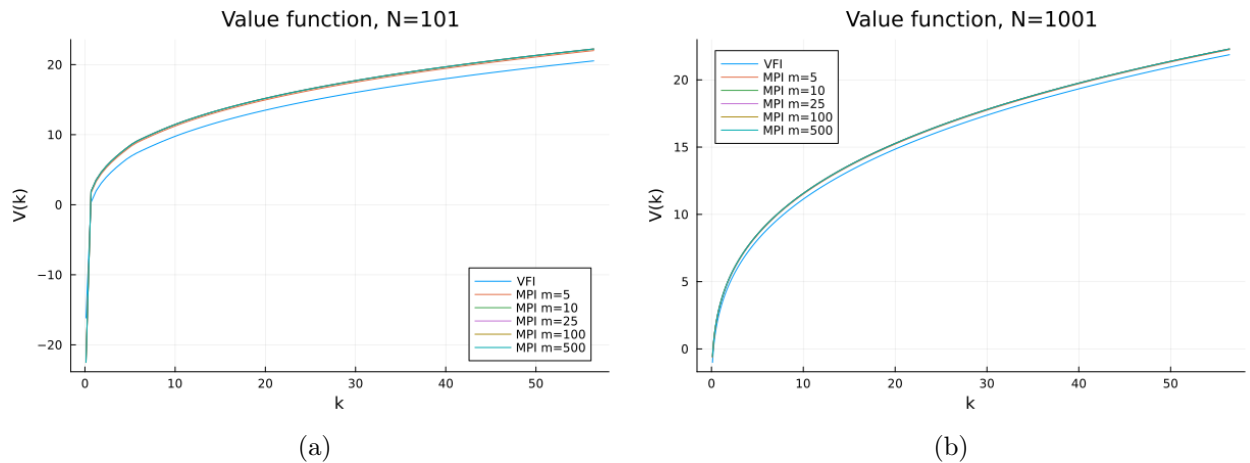


Figure 1: $V(k)$ for Howard's Modified Policy Iteration for k grid size of 101 and 1001

As well as the policy functions

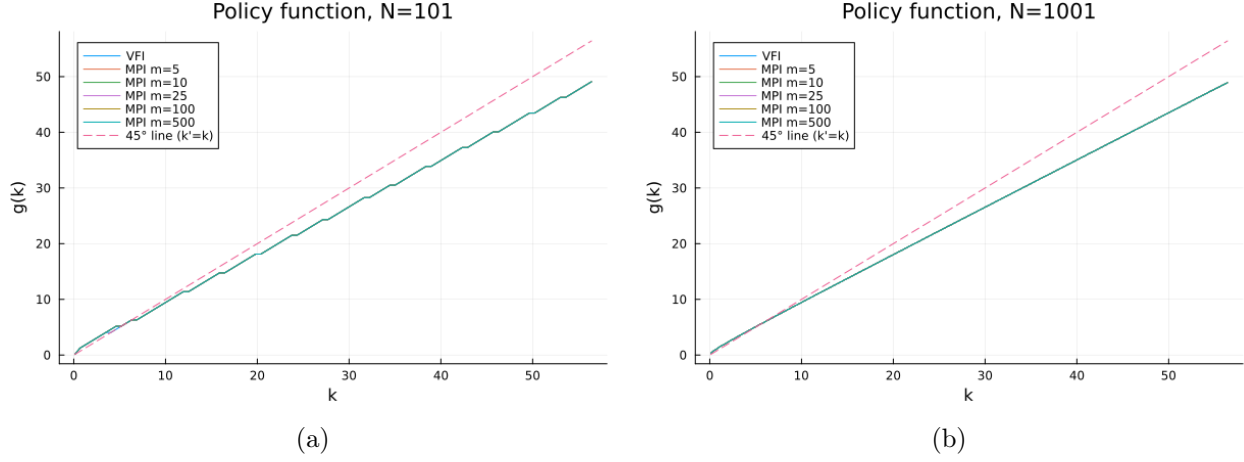


Figure 2: $g(k)$ for Howard's Modified Policy Iteration for k grid size of 101 and 1001

We can see that Howard's MPI produces very similar results for the Value and Policy functions for different values of m . Runtimes tend to be significantly shorter with Howard's MPI with greater m values for lower numbers of the k grid and similar runtimes for higher grid sizes.

Question b

*Now solve the model using MacQueen-Porteus bounds acceleration method (but no MPI). Apply the bounds in (1) every iteration, and (2) every 5 iterations.

1. Plot both the value function and decision rule in each case (each m). Do they all look the same? Comment on any difference.
2. Compare the convergence times in (1) and (2) to each other (and for each m) as well as to basic VFI and MPI. Prepare a well-organized table so as to make comparisons easier.

```
function mqp_vfi(; beta=0.96, A=1.0, alpha=0.4, delta=0.1,
    N=101, k_low=0.02, k_high=10.0,
    tol=1e-6, maxit=500000,
    apply_every=1,
    verbose=false)

# Step 0: grid
kstar, kmin, kmax, kgrid = kstar_and_grid(; beta=beta, A=A, alpha=alpha,
    delta=delta,
    k_low=k_low, k_high=k_high, N=N)

@assert length(kgrid) == N

if verbose
    @show first(kgrid) last(kgrid) step(kgrid)
end

# Step 1: initial guess V0 (zero function)
V = zeros(Float64, N)
V_new = similar(V)

policy_idx = collect(1:N) #start from k' = k on the grid
policy_idx_new = similar(policy_idx)

it = 0
converged = false

while it < maxit
    it += 1

    # Step 2: Bellman update (maximization + evaluation)
    dist = 0.0

    for i in 1:N
        k = kgrid[i]

        best_val = -Inf
        best_j = 1

        # scan all candidate k' on grid
        for j in 1:N
            kp = kgrid[j]
            c = f(k; A=A, alpha=alpha, delta=delta) - kp
            val = u(c) + beta * V[j]
        end
    end
end
```

```

        if val > best_val
            best_val = val
            best_j = j
        end
    end

    policy_idx_new[i] = best_j
    V_new[i] = best_val

    # decision rule distance (compare new policy vs old policy)
    dist = max(dist, abs(kgrid[best_j] - kgrid[policy_idx[i]]))
end

# MQP bounds acceleration
if apply_every > 0 && (it % apply_every == 0)
    cn = Inf
    cbar = -Inf
    for i in 1:N
        d = V_new[i] - V[i]
        cn = min(cn, d)
        cbar = max(cbar, d)
    end
    adj = 0.5 * (cbar + cn) / (1 - beta)
    for i in 1:N
        V_new[i] += adj #acceleration
    end
end

# update for next iteration
V .= V_new
policy_idx .= policy_idx_new

# stopping rule: decision rule convergence
if dist <= tol
    converged = true
    if verbose
        println("STOP at it=$it with dist=$dist")
    end
    break
end

if verbose && (it % 50 == 0)
    println("it=$it, dist=$dist, policy_idx[1:5]=", policy_idx[1:5])
end
end

if it == maxit
    converged = false
    if verbose
        println("Hit maxit=$maxit without convergence.")
    end
end

g = kgrid[policy_idx]
return (; V, g, kgrid, policy_idx, it, converged, kstar, kmin, kmax)
end

```

Value and Policy Function Comparison:

Figures 3 and 4 compare the MacQueen–Porteus (MQP) bounds acceleration method under two settings: applying the bounds every iteration ($m = 1$) and applying them every 5 iterations ($m = 5$), for two grid sizes ($N = 101$ and $N = 1001$). Overall, the plots show that both MQP settings lead to almost the same final solution.

For the value function $V(k)$, the curves for $m = 1$ and $m = 5$ are nearly indistinguishable for both grid sizes. This suggests that MQP bounds do not change the economic solution itself; instead, they mainly affect the speed of convergence during the iterations. The shape of the value function is also sensible: it increases with k and becomes flatter as k grows, which reflects diminishing marginal returns to capital under log utility.

For the decision rule (policy function) $g(k)$, the $m = 1$ and $m = 5$ lines also overlap almost perfectly. The policy function is increasing in k , meaning that higher current capital leads to higher next-period capital choices. The policy lies below the 45-degree line ($k' = k$) for most of the range, implying that the agent consumes part of current resources rather than keeping capital constant. The step-like pattern is expected because the model restricts k' to discrete grid points, so the optimal choice moves in small jumps instead of forming a perfectly smooth curve.

Comparing $N = 101$ and $N = 1001$, the policy function appears smoother with the finer grid, which is expected because a larger grid provides a closer approximation to the continuous problem. The fact that the MQP solutions look stable across both grid sizes supports that the numerical method is producing consistent results.

In summary, these plots indicate that applying MQP bounds every iteration or every five iterations produces essentially the same value function and decision rule. The main role of MQP is acceleration (potentially reducing convergence time), not changing the final computed policy.

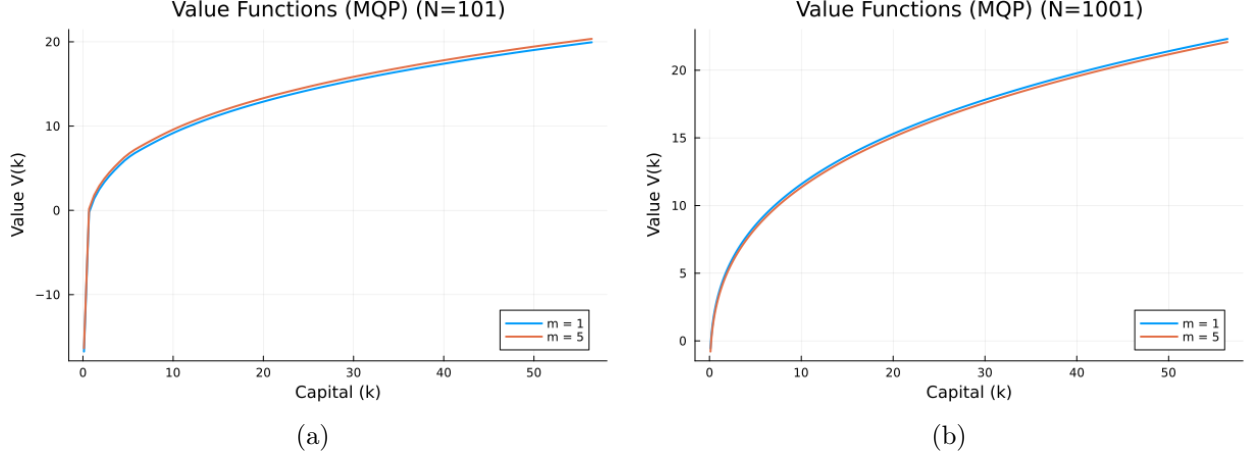


Figure 3: $V(k)$ for Howard's MQP for k grid size of 101 and 1001

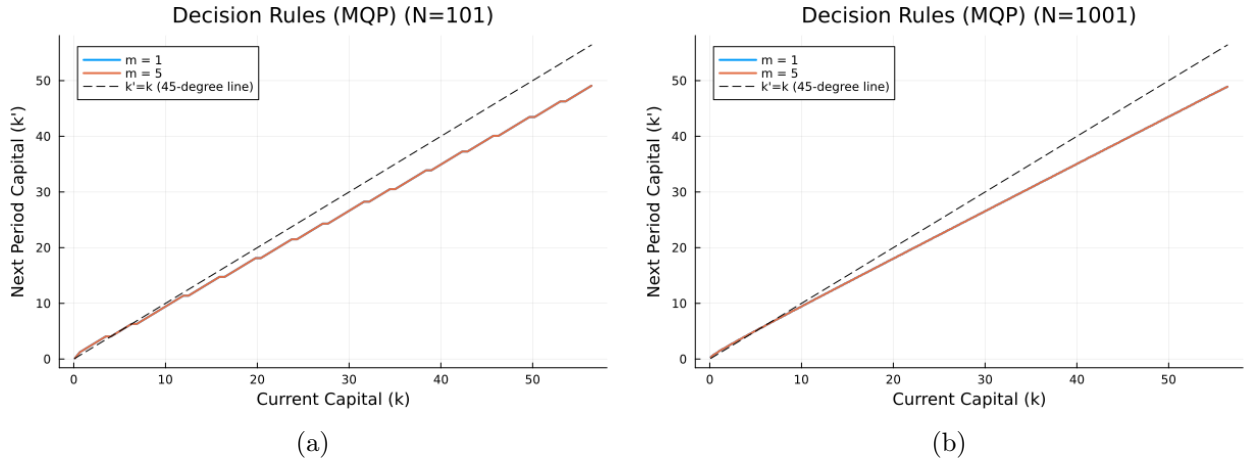


Figure 4: $g(k)$ for Howard's MQP for k grid size of 101 and 1001

Runtime and convergence comparison:

Table 2 compares convergence time and iteration counts for four approaches: basic value function iteration (VFI), Howard's Modified Policy Iteration (MPI) with different numbers of Howard steps m , and the MacQueen–Porteus (MQP) bounds method with two frequencies (apply every iteration vs. every 5 iterations). All methods use the same stopping rule based on decision rule (policy) convergence with tolerance 10^{-6} .

For basic VFI, runtime increases sharply when the grid becomes finer (from $N = 101$ to $N = 1001$). This is expected because each VFI iteration performs a full grid search over candidate choices, so the computational cost grows quickly with N . The number of iterations also increases (31 to 64), but the main driver of the time increase is the heavier work per iteration.

MPI is substantially faster than basic VFI, especially for the large grid. For $N = 1001$, MPI reduces the number of outer iterations from 64 to about 20–23, and total runtime drops from about 3.54 seconds to about 1.13–1.30 seconds, corresponding to a speedup of roughly $2.7\text{--}3.1\times$. The reason is that Howard steps carry out repeated *policy evaluation* cheaply (without a full grid search), which helps the value function update more fully between policy improvements. However, increasing m too much is not always better: when m becomes large (e.g., 100 or 500), the extra evaluation work adds overhead and total runtime rises again. In our results, $m = 10$ or $m = 25$ delivers the best overall performance.

For MQP bounds, both runtime and iteration counts are almost the same as basic VFI. This is because our stopping rule is based on policy convergence, while MQP mainly adjusts the level of the value function (approximately a constant shift) and typically does not change the policy update in a meaningful way. As a result, MQP does not reduce the number of policy iterations, and total time remains close to VFI. Applying bounds every iteration versus every 5 iterations also leads to only minor differences under a policy-based stopping rule.

Overall, MPI provides the largest runtime improvement, with a clear trade-off in the choice of m : small m may not accelerate enough, while very large m increases overhead. MQP bounds have limited impact on runtime in this setup because convergence is defined by the decision rule rather than the value function.

Table 2: Performance Comparison of VFI, MPI, and MQP

Method	Setting	Iterations	Time (seconds)	Speedup compared to baseline VFI
N = 101				
VFI	baseline	31	0.01626	1.000
MPI	m=5	17	0.00944	1.723
MPI	m=10	15	0.00881	1.845
MPI	m=25	14	0.00949	1.713
MPI	m=100	14	0.01651	0.985
MPI	m=500	14	0.05306	0.306
MQP	apply every=1	31	0.01604	1.014
MQP	apply every=5	31	0.01606	1.012
N = 1001				
VFI	baseline	64	3.37342	1.000
MPI	m=5	23	1.21148	2.785
MPI	m=10	20	1.06081	3.180
MPI	m=25	20	1.08209	3.118
MPI	m=100	20	1.18046	2.858
MPI	m=500	20	1.70180	1.982
MQP	apply every=1	64	3.34886	1.007
MQP	apply every=5	64	3.34457	1.009